LAPORAN KELOMPOK

EscrowChain - Rancang Bangun Sistem Rekening Bersama (Escrow) Berbasis Smart Contract Ethereum untuk Meningkatkan Kepercayaan Transaksi Jual Beli UMKM

Identitas Kelompok

Nama Kelompok:
Anggota:
1. 26.51.1009 ZAYYINI `ALIMATUS SA`IDAH
2. 26.51.1010 PARNO HUSNI
3. 26.51.1011 M. REZA D. NUR
4. 26.51.1019 WAHYUDI

Mata Kuliah:BLOKCHAIN AND FINTECH
Dosen Pengampu: Dhani Ariatmanto, S.Kom., M.Kom., Ph.D.
Tanggal Pengumpulan: 15/10/2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pertumbuhan transaksi jual beli daring mendorong banyak UMKM memasarkan produknya lewat media sosial, aplikasi pesan, dan marketplace[1]. Namun, transaksi di luar marketplace besar sering kali berlangsung tanpa jaminan keamanan dana. Pembeli khawatir sudah membayar tetapi barang tidak dikirim, sedangkan penjual khawatir barang sudah dikirim tetapi pembayaran tidak dilakukan[2].

Salah satu solusi yang umum dipakai adalah rekening bersama, yakni dana ditahan pihak ketiga sampai barang diterima. Namun, rekber konvensional bergantung pada kepercayaan terhadap perantara. Ada biaya jasa, proses manual, dan risiko penyalahgunaan dana oleh penyedia rekber itu sendiri.

Teknologi blockchain, khususnya smart contract pada Ethereum, memungkinkan logika dijalankan otomatis, transparan, dan tidak dapat diubah sepihak setelah di-deploy. Dana dikunci di dalam kontrak dan hanya dilepas jika syarat terpenuhi, seperti konfirmasi penerimaan barang, sehingga kepercayaan beralih dari perantara ke kode yang dapat diaudit[3].

Meski demikian, penerapan blockchain pada UMKM masih menghadapi tantangan, seperti kerumitan penggunaan dompet crypto, biaya gas, dan keamanan kontrak[4]. Karena itu, diperlukan rancangan sistem escrow yang sederhana, aman, dan sesuai kebutuhan UMKM. Berdasarkan hal tersebut, penelitian ini merancang dan membangun EscrowChain, sistem rekening bersama berbasis smart contract Ethereum untuk meningkatkan kepercayaan transaksi jual beli UMKM.

1.2 Tujuan

Tujuannya adalah:

a. Merancang arsitektur dan alur transaksi EscrowChain berbasis smart contract Ethereum untuk transaksi jual beli UMKM.
b. Membangun smart contract EscrowChain Solidity dan tampilan web untuk penjual dan pembeli.
c. Meningkatkan transparansi transaksi, mengurangi risiko penyalahgunaan dana, dan mendukung kepercayaan antara pembeli dan penjual.

1.3 Deskripsi Sistem

Nama Sistem: EscrowChain

Deskripsi Umum: EscrowChain merupakan sistem rekening bersama digital yang memanfaatkan teknologi blockchain untuk mengelola pembayaran dalam transaksi jual beli UMKM. Sistem ini menggunakan *smart contract* yang dikembangkan dengan bahasa pemrograman Solidity dan memanfaatkan pustaka OpenZeppelin untuk mendukung implementasi kontrak yang lebih terstruktur dan aman.EscrowChain dirancang untuk menahan dana pembeli sementara hingga persyaratan transaksi yang telah disepakati terpenuhi. Setelah pembeli mengonfirmasi penerimaan barang sesuai mekanisme yang ditetapkan, smart contract dapat meneruskan dana kepada penjual. Dengan mekanisme tersebut, sistem diharapkan dapat mengurangi risiko penyalahgunaan dana, meningkatkan transparansi proses pembayaran, dan mendukung kepercayaan antara pembeli dan penjual.

Pengembangan dan pengujian smart contract dilakukan menggunakan Remix IDE, sedangkan MetaMask digunakan sebagai wallet digital untuk menghubungkan pengguna dengan jaringan blockchain. Sistem dikembangkan dan diuji pada jaringan Ethereum Sepolia (*testnet*) sehingga proses transaksi dapat disimulasikan menggunakan aset uji tanpa menggunakan ETH bernilai nyata.

Target Pengguna:

1) Pembeli (Buyer): Pengguna yang melakukan pembelian produk UMKM, menghubungkan wallet MetaMask, menyetorkan dana ke smart contract, memantau status transaksi, dan mengonfirmasi penerimaan barang.
2) Penjual UMKM (Seller)**:** Pelaku usaha yang menawarkan produk, menyetujui ketentuan transaksi, mengirimkan barang kepada pembeli, dan menerima pembayaran setelah persyaratan transaksi terpenuhi.
3) Administrator Sistem (Admin)**:** Pihak yang membantu pengelolaan operasional dan pemantauan sistem serta menangani kendala sesuai kewenangan yang dirancang. Hak administrator terhadap dana dan transaksi dibatasi oleh aturan smart contract.

   Platform Blockchain: Ethereum, diuji pada testnet Sepolia. Bahasa kontrak Solidity ^0.8.20, pengembangan dengan Remix IDE, wallet MetaMask, antarmuka dengan ethers.js, dan verifikasi transaksi lewat Etherscan Sepolia.

   Alasan Pemilihan Platform:(1)Ethereum memiliki mesin eksekusi kontrak (EVM) yang terdefinisi jelas serta ekosistem dokumentasi dan peralatan yang luas (Buterin, 2014; Wood, 2014); (2) testnet Sepolia gratis dan memang disediakan untuk pengujian tanpa nilai ekonomi (Ethereum.org, n.d.); (3) setiap transaksi dan event dapat diverifikasi publik sehingga cocok sebagai bukti pengujian; (4) tersedia panduan keamanan dan pustaka teruji (Solidity, n.d.; OpenZeppelin, n.d.).

BAB II IDENTIFIKASI AKTOR SISTEM

2.1 Daftar Aktor

Sistem melibatkan tiga aktor manusia dan dua aktor sistem. Frontend web dan MetaMask berperan sebagai alat bantu bagi aktor manusia untuk menandatangani dan mengirim transaksi, bukan sebagai aktor yang memiliki kewenangan sendiri.

a. Pembeli: pemesan sekaligus pemilik dana yang ditahan.
b. Penjual: penyedia dan pengirim barang.
c. Arbiter: pihak netral yang memutuskan sengketa.
d. Smart Contract EscrowChain: aktor sistem yang menahan dana dan menegakkan aturan.
e. Jaringan Ethereum Sepolia (node dan validator): aktor sistem yang memvalidasi, mengurutkan, dan menyimpan transaksi.

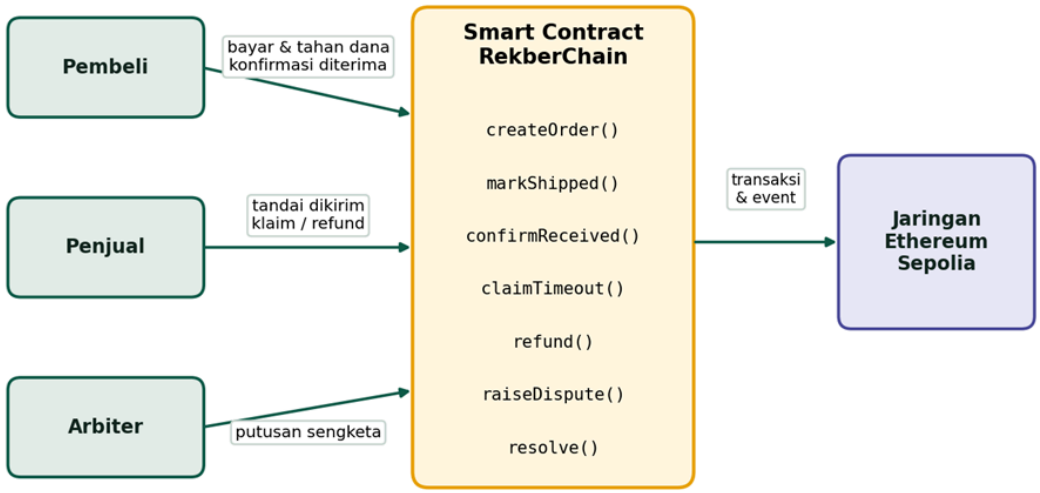
2.2 Deskripsi Aktor

No	Aktor	Deskripsi Peran	Hak Akses	Interaksi
1	Pembeli	Pihak yang memesan dan membayar barang; pemilik dana yang ditahan.	createOrder (deposit); confirmReceived; raiseDispute saat status Shipped; refund bila penjual tidak mengirim hingga batas waktu.	Mengirim ETH ke kontrak, mengonfirmasi penerimaan, menerima pengembalian dana.
2	Penjual	Pihak yang menyediakan dan mengirim barang.	markShipped; refund (membatalkan pesanan); raiseDispute; claimTimeout bila pembeli tidak merespons hingga batas waktu.	Mengirim barang (off-chain), menerima dana saat pesanan selesai.
3	Arbiter	Pihak netral yang ditunjuk saat pesanan dibuat.	resolve hanya untuk pesanan berstatus Disputed. Tidak dapat menarik dana untuk dirinya.	Memutuskan dana ke penjual atau dikembalikan ke pembeli.

4	Smart Contract RekberChain	Aktor sistem: menahan dana, mengatur perubahan status, dan mencatat event.	Mengelola saldo pesanan. Tidak ada akun admin yang dapat menarik dana.	Menerima panggilan fungsi, mentransfer ETH, memancarkan event OrderCreated dan StatusChanged.
5	Jaringan Ethereum Sepolia	Node dan validator yang memproses transaksi.	Memvalidasi dan mencatat transaksi yang sah; tidak dapat mengubah logika kontrak.	Menerima transaksi dari MetaMask, memberi hash dan konfirmasi blok.

2.3 Diagram Interaksi Aktor

Gambar 2.1 menunjukkan interaksi tiap aktor dengan fungsi smart contract. Pembeli dan penjual memanggil fungsi sesuai tahapan transaksi, arbiter hanya masuk saat terjadi sengketa, dan semua panggilan dicatat sebagai transaksi pada jaringan Sepolia.



Gambar 2.1 Diagram Interaksi Aktor dan Smart Contract

BAB III PERANCANGAN SMART CONTRACT

3.1 Identifikasi Kebutuhan

Smart contract dibutuhkan pada bagian sistem yang menuntut kepercayaan antarpihak dan tidak boleh dikendalikan satu pihak:

a. Penahanan dana (custody): dana pembeli dipegang kontrak, bukan operator atau penjual.
b. Mesin status: perubahan Funded, Shipped, Disputed, Completed, atau Refunded hanya terjadi lewat fungsi dan pelaku yang berwenang.
c. Pelepasan dana bersyarat: dana berpindah hanya setelah syarat terpenuhi (konfirmasi, putusan arbiter, atau lewat batas waktu).
d. Penegakan batas waktu: pembeli dan penjual terlindungi dari pihak lawan yang diam.
e. Jejak audit: setiap perubahan status memancarkan event yang tersimpan permanen.

Bagian yang tidak perlu on-chain adalah tampilan antarmuka, komunikasi, serta bukti pengiriman seperti nomor resi atau foto. Kontrak tidak dapat memverifikasi kondisi barang fisik; hal ini ditangani melalui konfirmasi pembeli dan keputusan arbiter, dan menjadi batasan yang dibahas pada Bab IV.

3.2 Deskripsi Smart Contract

No	Nama Contract	Fungsi	Input	Output	Aktor
1	EscrowChain	createOrder: membuat pesanan dan menahan dana (status Funded).	seller, arbiter, desc, daysToDeadline, nilai ETH (msg.value)	ID pesanan; event OrderCreated	Pembeli
2	EscrowChain	markShipped: menandai barang dikirim (Funded ke Shipped).	id	Status Shipped; event StatusChanged	Penjual
3	EscrowChain	confirmReceived: pembeli menyetujui, dana ke penjual.	id	ETH ke penjual; status Completed	Pembeli

4	EscrowChain	claimTimeout: penjual mengklaim dana bila pembeli tidak merespons setelah batas waktu.	id	ETH ke penjual; status Completed	Penjual
5	EscrowChain	refund: mengembalikan dana saat status Funded.	id	ETH ke pembeli; status Refunded	Penjual (kapan saja); Pembeli (setelah batas)
6	EscrowChain	raiseDispute: membuka sengketa (Shipped ke Disputed).	id	Status Disputed	Pembeli atau Penjual
7	EscrowChain	resolve: putusan arbiter.	id, toSeller (bool)	ETH ke penjual (Completed) atau ke pembeli (Refunded)	Arbiter
8	EscrowChain	_pay (private): transfer dana aman, dipanggil fungsi lain.	id, penerima, status akhir	ETH terkirim; saldo pesanan menjadi 0	Internal kontrak
9	EscrowChain	orders, orderCount (view): membaca data pesanan.	id	Data pesanan; jumlah pesanan	Semua pihak

3.3 Pseudo-code

Pseudo-code fungsi inti EscrowChain:

```
FUNCTION createOrder(seller, arbiter, desc, days) PAYABLE
  REQUIRE msg.value > 0
  REQUIRE seller != 0 AND seller != msg.sender
  REQUIRE arbiter != 0 AND arbiter != msg.sender AND arbiter != seller
  id <- orderCount + 1
  orders[id] <- {buyer: msg.sender, seller, arbiter, amount: msg.value,
                 deadline: now + days, status: FUNDED, desc}
  EMIT OrderCreated(id); RETURN id

FUNCTION confirmReceived(id)
  IF msg.sender == orders[id].buyer AND orders[id].status == SHIPPED
  THEN _pay(id, orders[id].seller, COMPLETED)
```

```
  ELSE REVERT "Tidak diizinkan"


FUNCTION refund(id)
  REQUIRE orders[id].status == FUNDED
  IF msg.sender == seller OR (msg.sender == buyer AND now > deadline)
  THEN _pay(id, buyer, REFUNDED)
  ELSE REVERT "Tidak diizinkan"


FUNCTION resolve(id, toSeller)
  REQUIRE msg.sender == arbiter AND status == DISPUTED
  IF toSeller THEN _pay(id, seller, COMPLETED)
  ELSE _pay(id, buyer, REFUNDED)


PRIVATE FUNCTION _pay(id, to, finalStatus)
  amt <- orders[id].amount
  orders[id].amount <- 0           // EFFECT dulu
  orders[id].status <- finalStatus  // EFFECT dulu
  EMIT StatusChanged(id, finalStatus)
  ok <- SEND amt TO to                  // INTERACTION terakhir
  REQUIRE ok
```

Pada _pay, saldo dan status diperbarui sebelum dana dikirim (pola checks-effects-interactions). Dengan begitu, pemanggilan ulang saat transfer berlangsung tidak dapat menarik dana dua kali (Solidity, n.d.).

3.4 Alur Eksekusi

Alur Kerja Smart Contract dari Awal sampai Selesai

Alur kerja dari awal sampai selesai:

1. **Persiapan.** Kontrak dideploy ke Sepolia. Pembeli, penjual, dan arbiter masing-masing memiliki akun MetaMask, lalu frontend dihubungkan dengan alamat kontrak.

2. **Pemesanan.** Pembeli memanggil createOrder dengan alamat penjual dan arbiter, batas waktu, dan sejumlah ETH. Dana masuk ke kontrak, status Funded.

3. **Pengiriman.** Penjual mengirim barang di luar rantai, lalu memanggil markShipped. Status menjadi Shipped.

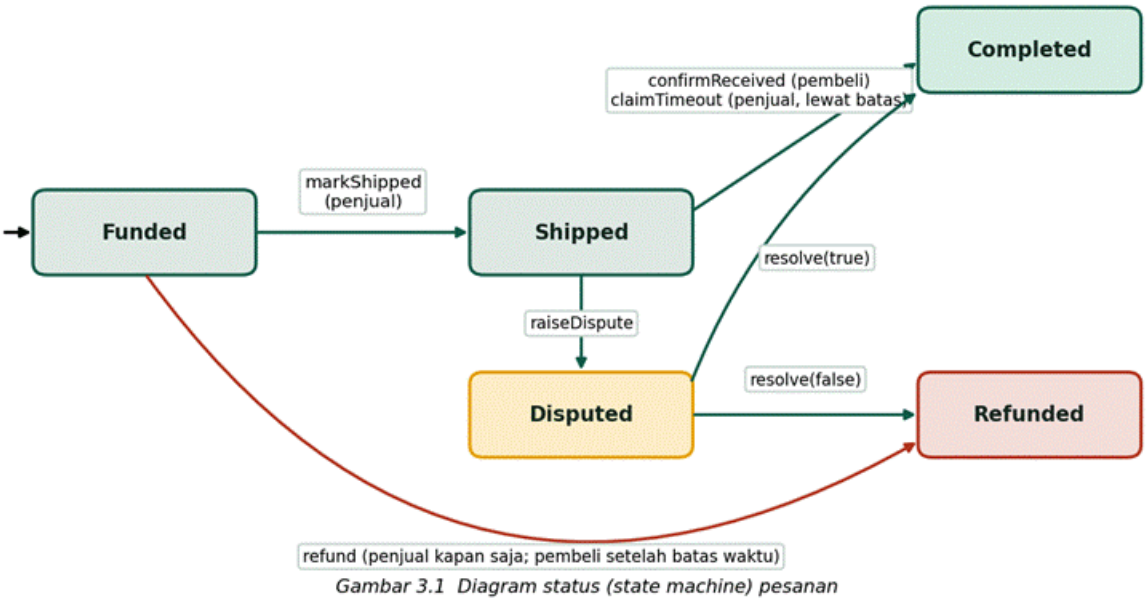
4. **Jalur normal.** Pembeli menerima barang dan memanggil confirmReceived. Dana dikirim ke penjual, status Completed.

5. **Pembeli tidak merespons.** Setelah batas waktu, penjual memanggil claimTimeout dan dana dikirim ke penjual.

6. **Pembatalan.** Penjual dapat memanggil refund saat status Funded. Pembeli dapat melakukannya setelah batas waktu bila barang tidak dikirim. Status Refunded.

7. **Sengketa.** Pembeli atau penjual memanggil raiseDispute pada status Shipped. Arbiter memanggil resolve untuk memenangkan salah satu pihak.

8. **Penutup.** Completed dan Refunded bersifat final. Setiap langkah menghasilkan hash transaksi yang dapat diperiksa di Etherscan Sepolia.



Gambar 3.1 Diagram status (state machine) pesanan

BAB IV ANALISIS KEAMANAN DAN RISIKO

4.1 Identifikasi Risiko

Risiko yang diidentifikasi pada rancangan EscrowChain:

R1. **Reentrancy.** Penerima dana berupa kontrak berbahaya dapat memanggil balik kontrak saat menerima ETH dan menarik dana berulang (Atzei et al., 2017; SWC Registry, n.d.).

R2. **Kontrol akses lemah.** Fungsi yang dapat dipanggil pihak yang tidak berhak, misalnya pembeli ikut menandai pengiriman.

R3. **Kesalahan input.** Alamat penjual atau arbiter salah ketik atau nilai ETH keliru. Transaksi blockchain tidak dapat dibatalkan.

R4. **Dana terkunci.** Arbiter kehilangan kunci privat atau tidak aktif, atau penerima menolak transfer sehingga _pay gagal terus-menerus.

R5. **Ketergantungan arbiter dan masalah oracle.** Kontrak tidak dapat memeriksa barang fisik, dan arbiter dapat berpihak atau berkolusi.

R6. **Ketergantungan timestamp.** Batas waktu memakai waktu blok yang dapat bergeser kecil oleh validator (Atzei et al., 2017).

R7. **Bug logika dan kontrak tidak dapat diubah.** Kesalahan yang lolos pengujian akan menetap setelah deploy.

R8. **Volatilitas dan keamanan wallet.** Nilai ETH berubah selama dana ditahan, dan kunci privat dapat dicuri lewat phishing.

R9. **Kepatuhan regulasi.** Rupiah wajib digunakan dalam transaksi pembayaran di Indonesia (UU No. 7 Tahun 2011, Pasal 21 ayat 1), dan PBI 18/40/PBI/2016 melarang penyelenggara jasa sistem pembayaran memproses transaksi dengan virtual currency. Aset kripto kini berada di bawah pengawasan OJK (POJK 27/2024). Karena itu prototipe ini hanya layak sebagai purwarupa akademik di testnet.

4.2 Mitigasi Risiko

No	Risiko	Mitigasi	Penerapan di EscrowChain	Rujukan
R1	Reentrancy	Pola checks-effects-interactions; ReentrancyGuard.	Diterapkan: saldo dan status diubah sebelum transfer di _pay. ReentrancyGuard disarankan sebagai lapisan tambahan.	Solidity (n.d.); OpenZeppelin (n.d.)
R2	Kontrol akses lemah	Pemeriksaan pengirim dan status pada tiap fungsi.	Diterapkan: require(msg.sender == ...) dan pemeriksaan status.	Atzei et al. (2017)
R3	Kesalahan input	Validasi input dan konfirmasi di UI.	Diterapkan: alamat nol, penjual sama pembeli, dan nilai 0 ditolak; UI memvalidasi format alamat.	Solidity (n.d.)
R4	Dana terkunci	Pull payment; arbiter berlapis atau cadangan.	Belum: dicatat sebagai saran pengembangan.	OpenZeppelin (n.d.)
R5	Arbiter dan oracle	Arbiter dipilih bersama; ke depan multisig atau staking.	Sebagian: arbiter ditentukan saat pesanan dibuat dan tidak dapat menarik dana.	Atzei et al. (2017)
R6	Timestamp	Gunakan skala hari, bukan detik.	Diterapkan: batas dihitung dalam hari.	Atzei et al. (2017)
R7	Bug dan immutability	Uji unit, analisis statis, audit, uji di testnet.	Sebagian: uji manual di Sepolia; uji otomatis disarankan.	Solidity (n.d.)
R8	Volatilitas dan wallet	Stablecoin; edukasi keamanan wallet.	Belum: memakai ETH testnet.	Buterin (2014)
R9	Regulasi	Rupiah digital berizin; kemitraan dengan penyelenggara berizin.	Dibatasi pada testnet dan tujuan akademik.	UU 7/2011; PBI 18/40/2016; POJK 27/2024

4.3 Studi Kasus

Kasus 1: The DAO (2016)

The DAO adalah kontrak dana investasi terdesentralisasi di Ethereum. Fungsi penarikannya mengirim ETH sebelum saldo pemanggil dikurangi, sehingga penyerang dapat memanggilnya berulang dan menguras sekitar 3,6 juta ETH. Peristiwa ini berujung pada hard fork Ethereum (Mehar et al., 2019; Atzei et al., 2017). **Pelajaran:** perbarui status sebelum mengirim dana (checks-effects-interactions), batasi nilai yang ditahan, dan lakukan audit. Pelajaran ini diterapkan langsung pada fungsi _pay.

Kasus 2: Parity Multi-sig Wallet (2017)

Pada November 2017, seorang pengguna memicu fungsi yang membuat library wallet bersama dihancurkan (self-destruct). Wallet multi-sig yang bergantung padanya tidak dapat lagi memindahkan dana, dan lebih dari 500 ribu ETH terkunci (Parity Technologies, 2017). **Pelajaran:** fungsi inisialisasi dan fungsi sensitif harus dilindungi kontrol akses, ketergantungan pada library eksternal perlu dikaji, dan kode yang tidak dapat diubah menuntut pengujian ketat. Risiko dana terkunci ini juga relevan bagi EscrowChain (R4).

BAB V PENUTUP

5.1 Kesimpulan

Analisis kelompok menunjukkan bahwa escrow merupakan kasus penggunaan smart contract yang tepat karena masalah intinya adalah kepercayaan antarpihak yang tidak saling kenal. EscrowChain menggantikan operator rekber dengan kontrak yang menahan dana, mengatur lima status pesanan, dan menegakkan batas waktu secara otomatis. Tiga aktor manusia dan dua aktor sistem teridentifikasi dengan hak akses yang dibatasi per fungsi dan status.

Analisis risiko menemukan sembilan risiko utama. Reentrancy dan kontrol akses sudah dimitigasi pada kode, sedangkan risiko dana terkunci, volatilitas, dan kepatuhan regulasi masih terbuka. Prototipe ini layak sebagai purwarupa akademik di testnet, bukan sebagai layanan pembayaran produksi.

5.2 Saran

- Menambahkan ReentrancyGuard dan pola pull payment (penerima menarik dananya sendiri) agar kegagalan transfer tidak mengunci pesanan.
- Memisahkan batas waktu pengiriman dan batas waktu konfirmasi pembeli. Pada rancangan sekarang hanya ada satu batas waktu dari pembuatan pesanan.
- Memakai token ERC-20 (stablecoin rupiah simulasi) untuk mengurangi risiko volatilitas.

- Menambah uji otomatis (Hardhat atau Foundry) dan analisis statis, serta mengukur biaya gas tiap fungsi dari transaksi di Sepolia.
- Memperkuat mekanisme arbiter (multi-arbiter atau multisig) dan menambah biaya platform sebagai model bisnis.
- Mengkaji lebih jauh kepatuhan terhadap ketentuan OJK, Bank Indonesia, dan perlindungan konsumen sebelum dipakai untuk transaksi riil.

DAFTAR PUSTAKA

Filipus, Y. N., Engel, M. M., & Witanto, E. N. (2025). Model reservasi hotel desentralisasi menggunakan smart contract ERC-1155 dan escrow di Optimistic Rollups. Edumatic: Jurnal Pendidikan Informatika, 9(3), 865–874. https://doi.org/10.29408/edumatic.v9i3.32620

Khaidir K.N, L., & Nasution, M. I. P. (2024). Analisis keamanan data terhadap penggunaan e-wallet sebagai alat transaksi digital untuk mencegah penipuan online. Jurnal Sains Student Research, 2(4), 108–116. https://doi.org/10.61722/jssr.v2i4.1955

Mandasari, R., Wulandari, I., Jaklin, V., & Rochadiani, T. H. (2024). Digitalisasi UMKM menuju masyarakat maju di Kampung Kalipaten. Society: Jurnal Pengabdian dan Pemberdayaan Masyarakat, 4(2), 149–156. https://doi.org/10.37802/society.v4i2.486

Purnomo, B. H., Rismayadi, D. A., & Thoriq, M. R. F. (2025). Adopsi blockchain sebagai solusi keamanan dan transparansi transaksi digital di industri fintech. Jurnal Minfo Polgan, 13(2), 2486–2492. https://doi.org/10.33395/jmp.v13i2.14523

Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A survey of attacks on Ethereum smart contracts (SoK). Dalam M. Maffei & M. Ryan (Eds.), Principles of Security and Trust (POST 2017), LNCS 10204 (hlm. 164–186). Springer. https://doi.org/10.1007/978-3-662-54455-6_8

Bisnis.com. (2025a, 2 Januari). Bos OJK pastikan peralihan pengawasan aset kripto dari Bappebti berjalan lancar. https://market.bisnis.com/read/20250102/94/1828373/bos-ojk-pastikan-peralihan-pengawasan-aset-kripto-dari-bappebti-berjalan-lancar

Bisnis.com. (2025b, 10 Januari). OJK mulai awasi aset kripto, ada 16 pedagang telah berizin. https://market.bisnis.com/read/20250110/94/1830658/ojk-mulai-awasi-aset-kripto-ada-16-pedagang-telah-berizin

Buterin, V. (2014). A next-generation smart contract and decentralized application platform (Ethereum white paper). https://ethereum.org/en/whitepaper/

Christidis, K., & Devetsikiotis, M. (2016). Blockchains and smart contracts for the Internet of Things. IEEE Access, 4, 2292–2303. https://doi.org/10.1109/ACCESS.2016.2566339

Ethereum.org. (n.d.). Networks (termasuk testnet Sepolia). https://ethereum.org/en/developers/docs/networks/

Mehar, M. I., Shier, C. L., Giambattista, A., Gong, E., Fletcher, G., Sanayhie, R., Kim, H. M., & Laskowski, M. (2019). Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack. Journal of Cases on Information Technology, 21(1), 19–32. https://doi.org/10.4018/JCIT.2019010102

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf

OpenZeppelin. (n.d.). OpenZeppelin Contracts documentation. https://docs.openzeppelin.com/contracts

Parity Technologies. (2017, 8 November). A postmortem on the Parity multi-sig library self-destruct. https://www.parity.io/blog/a-postmortem-on-the-parity-multi-sig-library-self-destruct/

Peraturan Bank Indonesia Nomor 18/40/PBI/2016 tentang Penyelenggaraan Pemrosesan Transaksi Pembayaran.

Peraturan Otoritas Jasa Keuangan Nomor 27 Tahun 2024 tentang Penyelenggaraan Perdagangan Aset Keuangan Digital Termasuk Aset Kripto.

Peraturan Pemerintah Nomor 49 Tahun 2024 tentang Peralihan Tugas Pengaturan dan Pengawasan Aset Keuangan Digital Termasuk Aset Kripto serta Derivatif Keuangan.

Smart Contract Weakness Classification Registry. (n.d.). SWC-107: Reentrancy. https://swcregistry.io/docs/SWC-107

Solidity. (n.d.). Security considerations. Solidity documentation. https://docs.soliditylang.org/en/latest/security-considerations.html

Szabo, N. (1997). Formalizing and securing relationships on public networks. First Monday, 2(9). https://doi.org/10.5210/fm.v2i9.548

Undang-Undang Nomor 4 Tahun 2023 tentang Pengembangan dan Penguatan Sektor Keuangan.

Undang-Undang Nomor 7 Tahun 2011 tentang Mata Uang.

Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger (Yellow paper). https://ethereum.github.io/yellowpaper/paper pdf.

LAMPIRAN

Lampiran A. Kode sumber smart contract EscrowChain.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;


contract EscrowChain {

    enum Status { Funded, Shipped, Disputed, Completed, Refunded }


    struct Order {

        address buyer; address seller; address arbiter;

        uint256 amount; uint256 deadline; Status status; string desc;

         }


    uint256 public orderCount;

    mapping(uint256 => Order) public orders;


    event OrderCreated(uint256 indexed id, address buyer, address seller, uint256 amount);

    event StatusChanged(uint256 indexed id, Status status);


    function createOrder(address seller, address arbiter, string calldata desc, uint256 daysToDeadline)

        external payable returns (uint256 id)

         {

        require(msg.value > 0, "Dana harus > 0");

        require(seller != address(0) && seller != msg.sender, "Penjual tidak valid");
```

```
    require(arbiter != address(0) && arbiter != msg.sender && arbiter != seller, "Arbiter tidak valid");
    id = ++orderCount;
    orders[id] = Order(msg.sender, seller, arbiter, msg.value,
        block.timestamp + daysToDeadline * 1 days, Status.Funded, desc);
    emit OrderCreated(id, msg.sender, seller, msg.value);
     }

function markShipped(uint256 id) external {
    Order storage o = orders[id];
    require(msg.sender == o.seller && o.status == Status.Funded, "Tidak diizinkan");
    o.status = Status.Shipped; emit StatusChanged(id, o.status);
     }

function confirmReceived(uint256 id) external {
    Order storage o = orders[id];
    require(msg.sender == o.buyer && o.status == Status.Shipped, "Tidak diizinkan");
    _pay(id, o.seller, Status.Completed);
     }

function claimTimeout(uint256 id) external {
    Order storage o = orders[id];
    require(msg.sender == o.seller && o.status == Status.Shipped, "Tidak diizinkan");
    require(block.timestamp > o.deadline, "Belum lewat batas");
    _pay(id, o.seller, Status.Completed);
     }

function refund(uint256 id) external {
    Order storage o = orders[id];
    require(o.status == Status.Funded, "Status salah");
```

```
        require(msg.sender == o.seller || (msg.sender == o.buyer && block.timestamp > o.deadline), "Tidak diizinkan");
        _pay(id, o.buyer, Status.Refunded);
         }

    function raiseDispute(uint256 id) external {
        Order storage o = orders[id];
        require(o.status == Status.Shipped && (msg.sender == o.buyer || msg.sender == o.seller), "Tidak diizinkan");
        o.status = Status.Disputed; emit StatusChanged(id, o.status);
         }

    function resolve(uint256 id, bool toSeller) external {
        Order storage o = orders[id];
        require(msg.sender == o.arbiter && o.status == Status.Disputed, "Tidak diizinkan");
        if (toSeller) _pay(id, o.seller, Status.Completed);
        else _pay(id, o.buyer, Status.Refunded);
         }

         // Pola checks-effects-interactions: status diubah sebelum transfer (anti-reentrancy)
    function _pay(uint256 id, address to, Status s) private {
        Order storage o = orders[id];
        uint256 amt = o.amount;
        o.amount = 0; o.status = s;
        emit StatusChanged(id, s);
        (bool ok, ) = payable(to).call{value: amt}("");
        require(ok, "Transfer gagal");
         }
}
```

Lampiran B. Tangkapan layar antarmuka

[Tempel di sini: tangkapan layar form buat pesanan, daftar pesanan, dan tiap status (Funded, Shipped, Completed, Disputed).]

Lampiran C. Bukti transaksi di Sepolia

[Isi setelah pengujian: alamat kontrak, hash transaksi tiap langkah (createOrder, markShipped, confirmReceived, refund, raiseDispute, resolve), tautan Etherscan, dan gas used per fungsi.]
